

Post Class Documentation

entityClasses.Post

Overview

Post represents one discussion-board post in the student discussion system. The class stores the core content of a post (header and body), authorship and timestamp metadata through CreateInfo, and solved/closed state through a boolean flag. Together, these elements support the basic entity behavior needed by the student-facing discussion workflow: create a post, read its content later, update it, show whether it has been resolved, and preserve the metadata needed for persistence and display.

TP2 Task 3 specifically requires the class to live in the entityClasses package, document its purpose and attributes using Javadoc, explain how the class supports CRUD and input validation, and append screenshots of internal comments after the Javadoc section.

Class Synopsis

```
package entityClasses;

public class Post {
    private CreateInfo postInfo;
    private String header;
    private String body;
    private boolean solved = false;
}
```

Aspect	Description
Superclass	java.lang.Object
Related type	CreateInfo (stores post ID, author, creation time, edit status, and edit time)
Primary responsibility	Represent one persisted post instance and expose the data/services needed by UI, controller, and database layers.
CRUD realization	Create via constructors, Read via getters/formatters, Update via editPost and markAsSolved, Delete by removing the Post object from the owning collection/database using its stable ID.
Validation role	Performs lightweight entity-level safeguards while assuming stronger trim/required/character checks are enforced by shared validators in upper layers.

Field Summary

Field	Type	Source / Rationale	Supports
postInfo	CreateInfo	Needed to centralize stable identity, author, create timestamp, and edit metadata. This prevents the Post class from duplicating cross-cutting metadata fields.	Create, Read, Update, persistence, audit-style display
header	String	Student discussion posts need a concise title/header for list views, search results, and visual scanning.	Create, Read, Update, summary displays
body	String	Stores the main post content shown in the detailed view and edited later by the author or permitted user.	Create, Read, Update
solved	boolean	Supports the resolved/closed workflow expected in discussion systems and anticipated staff/student follow-up handling.	Update, status display, future close/reopen actions

Constructor Summary

Constructor	Purpose
Post(String user, String header, String body)	Creates a brand-new post from runtime user input and initializes CreateInfo for authorship and creation time.
Post(CreateInfo info, String header, String body, boolean solved)	Rebuilds an existing post from persistent storage, preserving metadata and solved status.

Method Summary

Method	Returns	Purpose
editPost(String newHeader, String newBody)	boolean	Updates one or both editable text fields and records that an edit occurred.
markAsSolved(boolean solved)	void	Changes resolved/open state.
getCreateDate()	String	Returns creation date as formatted text.
getID()	String	Returns stable post ID.
getEdited()	boolean	Returns whether the post has been edited.
getEditedTime()	String	Returns last-edit timestamp as formatted text.
getUser()	String	Returns the author username.
getHeader()	String	Returns header text.
getBody()	String	Returns body text.
getSolved()	boolean	Returns solved state.
getSolvedText()	String	Returns " (SOLVED)" when solved, else empty string.
getCreateInfoFormatted()	String	Formats create and edit metadata for UI display.

CRUD and Input-Validation Mapping

Area	How it is realized	Relevant code elements	Notes
Create	A new Post object is created from user input or reconstructed from saved data.	Both constructors	CreateInfo is initialized or preserved so the object has author, ID, and timestamp metadata immediately.
Read	The object exposes stored values for UI tables, detail screens, persistence, and formatted display.	Getter methods + getCreateInfoFormatted()	Separate getters keep presentation logic outside the entity while still making the entity easy to inspect.
Update	Editable content and solved/open state can change after creation.	editPost(), markAsSolved()	The edit method updates timestamps through CreateInfo.edit() so change history is not lost.
Delete	Deletion is performed by the owner of the collection/database, which removes the Post instance using its ID.	getID() + external collection/DB layer	The entity does not delete itself; instead it provides stable identity so higher layers can safely remove it.
Validation	The class rejects an all-empty edit request and avoids overwriting fields with empty replacement strings.	editPost()	Broader trim/required/character/max-length validation is expected from shared validators in controller/UI layers, consistent with the team validation rules.

Why the validation note matters: the team's earlier validation document calls for trimming, required-field checks, character restrictions, and shared validation helpers across layers. This Post class contributes an entity-level safeguard against a meaningless empty update, while the rest of the stronger input policy is expected to live in the higher-level validation and controller code.

Field Detail

postInfo

```
private CreateInfo postInfo
```

Holds the metadata that gives the post a stable identity and traceability. In practice, this lets the rest of the system refer to a post by ID, show who created it, show when it was created, and record later edits without scattering those responsibilities across multiple string/boolean fields.

header

```
private String header
```

Stores the short title or headline text shown in list displays and detail views. Keeping the header separate from the body supports fast scanning in tables and makes future features such as search results, summaries, or sorting easier to implement.

body

```
private String body
```

Stores the main content of the post. This is the primary user-authored text that is displayed when a post is opened and the value most likely to be edited later.

solved

```
private boolean solved = false
```

Represents whether the post has been resolved/closed. This supports current student workflows and also anticipates likely staff workflows, such as marking a thread resolved after guidance has been given.

Constructor Detail

Post(String user, String header, String body)

```
Post(String user, String header, String body)
```

- Purpose: create a brand-new discussion post from runtime inputs.
- Rationale: when a student creates a post, the class must immediately capture authorship and creation metadata, not just the header/body content.
- How it helps the user stories: it gives the application one compact object that can be inserted into collections, saved to the database, and shown in UI lists or detail windows.

Post(CreateInfo info, String header, String body, boolean solved)

```
Post(CreateInfo info, String header, String body, boolean solved)
```

- Purpose: reconstruct a post that already exists in persistent storage.
- Rationale: a discussion system must be able to load posts back from SQL/database storage without losing their original ID, timestamps, author, or solved state.
- How it helps the user stories: supports reloading saved posts, preserving history, and showing consistent status after application restart or later navigation.

Method Detail

boolean editPost(String newHeader, String newBody)

```
boolean editPost(String newHeader, String newBody)
```

- Purpose: update the editable text of a post without creating a new object.
- Behavior: returns false when both candidate replacements are empty strings; otherwise updates only the fields that were given non-empty replacements.
- Why the code is written this way: partial edits are allowed, so a caller can change just the header or just the body without having to resupply the unchanged field.
- Metadata side effect: calls postInfo.edit() so the edit flag and edit timestamp remain accurate.

void markAsSolved(boolean solved)

```
void markAsSolved(boolean solved)
```

- Purpose: toggle the resolved/open state of the post.
- How it helps the workflow: resolved state is a common discussion-board behavior and supports future filtering, badges, close/reopen actions, and staff moderation flows.

String getCreateInfoFormatted()

```
String getCreateInfoFormatted()
```

- Purpose: produce a human-readable metadata string for UI display.
- Behavior: returns the creation date alone for untouched posts, and appends edited-date information when the post has been changed.
- Why this helper exists: it keeps repetitive formatting logic out of the UI/controller layer while still leaving the raw underlying metadata inside CreateInfo.

How the Current Design Enables Student Stories and Likely TP3 Extensions

Need / story area	Current Post support	Why the design helps going forward
Student creates and views posts	Constructors + getters + metadata accessors	A post can be created once and then displayed consistently in table, detail, and persistence contexts.
Student edits a post	editPost() + edit metadata via CreateInfo	The object preserves original identity and create time while still recording that a later edit happened.
Student sees whether a thread is closed/resolved	solved, getSolved(), getSolvedText(), markAsSolved()	The boolean state is simple now, but it is enough to drive labels, filters, and future role-based restrictions.
Persistence / reload	Full constructor + getID()	Objects can be reconstructed from storage while keeping the same identity and status.
Future staff-driven resolution workflows	solved flag + stable metadata	A later staff epic could attach resolution notes, resolver identity, or close reason without changing the basic role of the class.
Delete / moderation lifecycle	Stable ID and metadata available to external owner layers	Higher layers can perform hard-delete or soft-delete policies while the entity remains responsible for representing one post instance.

Likely future additions, if TP3 or instructional feedback requires them, could include a close reason, tags/category, staff-resolution note, visibility flags, or a soft-delete flag. The current design does not implement those yet, but it already separates identity/metadata from content, which makes those extensions more manageable.

Screenshots of Key Internal Comments and Code

The screenshots below are placed after our Javadoc-style documentation and show internal comments and code sections that clarify requirement coverage and non-obvious implementation choices.

Screenshot 1 - Package placement, class Javadoc, and attribute declarations

```
1 package entityClasses;
2
3 /**
4  * <p>
5  * Title: Post
6  * </p>
7  *
8  * <p>
9  * Description: This class is a container for post entities within the system, it contains all relevant post information including
10 * CreateInfo, header, body, and solved status. It also contains several helper functions to utilize this data in accordance with
11 * the services provided in the application.
12 * </p>
13 *
14 *
15 * @author Sutton Harr
16 * @version 1.1
17 * @see CreateInfo
18 *
19 *
20 */
21 public class Post {
22     private CreateInfo postInfo; //Info class to store information about the post
23     private String header; //The header text of the post
24     private String body; //The body text of the post
25     private boolean solved = false; //To Determine if the post is solved
26
27 }
```

This screenshot shows that the class is placed in the entityClasses package and that the source already contains a class-level Javadoc block. It also shows the field comments explaining why postInfo, header, body, and solved exist.

Screenshot 2 - Constructors for new and persisted posts

```
27
28 /**
29  * Constructor for new post
30  *
31  * @param user the user who created the post
32  * @param header the header of the post
33  * @param body the body text of the post
34  */
35 public Post(String user, String header, String body)
36 {
37     postInfo = new CreateInfo(user); //Creates user info object
38     this.header = header; //Stores the input header
39     this.body = body; //Stores the input body
40 }
41
42
43 /**
44  * Full constructor method for building from SQL Save
45  *
46  * @param info the relevant create info of the post
47  * @param header the header of the post
48  * @param body the body of the post
49  * @param solved the solved status of the post
50  */
51 public Post(CreateInfo info, String header, String body, boolean solved)
52 {
53     this.postInfo = info;
54     this.header = header;
55     this.body = body;
56     this.solved = solved;
57 }
58 }
```

This screenshot highlights the two-constructor design: one path for a newly created runtime post and one path for rebuilding a post from SQL/database state. That split is important because persisted objects must keep their original metadata and solved state.

Screenshot 3 - Update logic and solved-state mutation

```
59  /**
60   * Edit post method, captures edit status
61   *
62   * @param newHeader the new header value of the post
63   * @param newBody the new body of the post
64   * @return a boolean based on if the post was changed
65   */
66  public boolean editPost(String newHeader, String newBody)
67  {
68      if (newHeader == "" && newBody == "") return false; //If both input are empty do not change the post, will return false
69
70      header = (newHeader != "") ? newHeader : header; //If no input for a new header then it will not change anything
71      body = (newBody != "") ? newBody : body; //If there is no input for a new body keep the body the same
72
73      postInfo.edit(); //Save latest post edit time
74      return true; //Return true for edit complete
75  }
76
```

This screenshot shows the editPost method and the inline comments describing partial updates and edit tracking. It also shows the separate solved-state mutator that supports resolve/close behavior without mixing it into the text-update routine.

Screenshot 4 - Solved-text and formatted create/edit metadata helpers

```
146  /**
147   * Gets the solved status of the post
148   *
149   * @return a boolean indicating solved status
150   */
151  public boolean getSolved() {return solved;}
152
153
154  /**
155   * Helper Method to determine what to display next to the header when a post is solved
156   *
157   * @return a string indicating the solved status of the post, empty if not marked as solved
158   */
159  public String getSolvedText() {if(solved) return " (SOLVED)"; else return "";}
160
161
162  /**
163   * Returns the complete creation info of the post (Create + Edit Dates)
164   * If the post has been edited return a string displaying last edit date else return nothing
165   *
166   * @return a String containing the formatted creation and edit dates
167   */
168  public String getCreateInfoFormatted()
169  {
170      if(postInfo.getEdited()) return getCreateDate() + " (Edited: " + postInfo.getLastEditDate() + ")";
171      else return getCreateDate();
172  }
173 }
```

This screenshot shows the helper methods that format solved-state messaging and create/edit metadata for the rest of the system. These helpers reduce repeated UI formatting code and keep status display consistent.